

CSA1718 AI
LAB PROBLEMS

SANJAI.S(192511018)

EXPERIMENT 9:

MAP COLORING USING CSP

AIM

To implement the Map Coloring Problem using Constraint Satisfaction Problem (CSP) and assign colors to each region such that no two adjacent regions have the same color.

ALGORITHM

1. Start.
2. Define the map as a graph.
3. Define the available colors (Red, Green, Blue).
4. Select an uncolored region.
5. Assign a color to the region.
6. Check whether any adjacent region has the same color.
7. If the color is valid, move to the next region.
8. Otherwise, try another color (Backtracking).
9. Repeat until all regions are colored.
10. Display the color assigned to each region.
11. Stop.

PSEUDOCODE

START

Create graph

Create list of colors

Initialize empty result

Function IS_SAFE(node, color)

For each adjacent node

If adjacent node has same color

Return False

Return True

Function SOLVE(index)

If all nodes are colored

Return True

For each color

If IS_SAFE(node, color)

Assign color

If SOLVE(next node)

Return True

Remove assigned color

Return False

Call SOLVE(0)

If solution exists

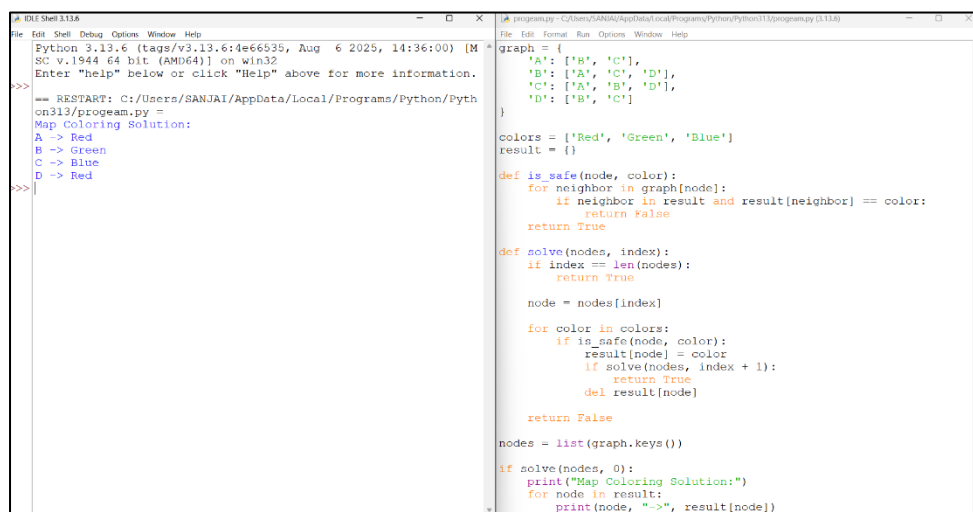
Print assigned colors

Else

Print "No Solution"

STOP

OUTPUT



```
Python 3.13.6 (tags/v3.13.6:4e66535, Aug 6 2025, 14:36:00) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

>>> == RESTART: C:/Users/SANJAI/AppData/Local/Programs/Python/Python313/progeam.py ==
Map Coloring Solution:
A -> Red
B -> Green
C -> Blue
D -> Red
>>>
```

```
graph = {
    'A': ['B', 'C'],
    'B': ['A', 'C', 'D'],
    'C': ['A', 'B', 'D'],
    'D': ['B', 'C']
}

colors = ['Red', 'Green', 'Blue']
result = {}

def is_safe(node, color):
    for neighbor in graph[node]:
        if neighbor in result and result[neighbor] == color:
            return False
    return True

def solve(nodes, index):
    if index == len(nodes):
        return True

    node = nodes[index]

    for color in colors:
        if is_safe(node, color):
            result[node] = color
            if solve(nodes, index + 1):
                return True
            del result[node]

    return False

nodes = list(graph.keys())

if solve(nodes, 0):
    print("Map Coloring Solution:")
    for node in result:
        print(node, "->", result[node])
```

Result

The Map Coloring Problem was successfully solved using the CSP Backtracking algorithm, and colors were assigned to all regions without violating the adjacency constraints.

EXPERIMENT 10:

TIC Tac Toe Game

AIM

To develop a Tic Tac Toe game in Python where two players play alternately until one player wins or the game ends in a draw.

ALGORITHM

1. Start.
2. Create an empty 3×3 game board.
3. Set the first player as 'X'.
4. Display the board.
5. Ask the current player to enter a position.
6. If the position is empty, place the player's symbol.
7. Check whether the player has won.
8. If a player wins, display the winner and stop.
9. Otherwise, switch to the other player.
10. Repeat until all 9 positions are filled.
11. If no player wins, declare the game as a draw.
12. Stop.

PSEUDOCODE

START

Create empty board

Set current player = X

Repeat 9 times

Display board

Read player position

If position is empty

Place player's symbol

Else

Ask for another position

Check winning combinations

If player wins

Display winner

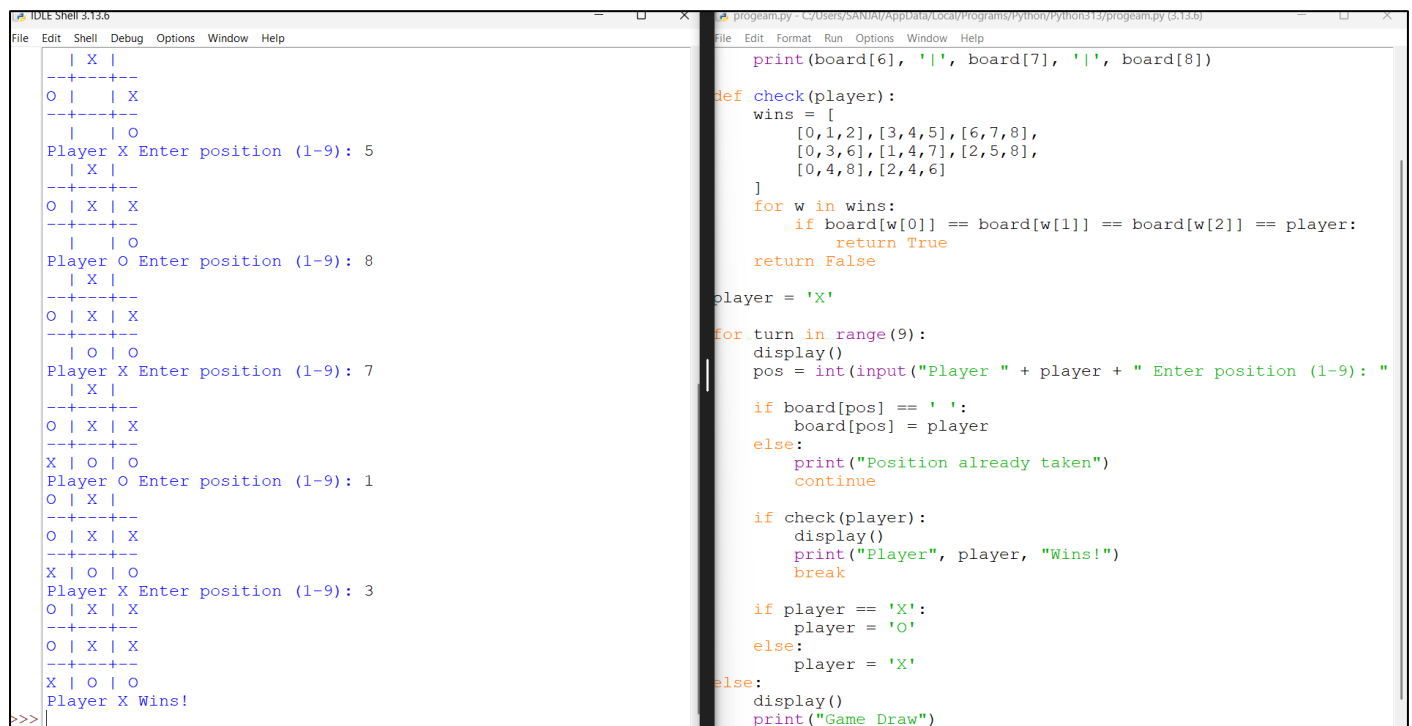
STOP

Change player

Display "Game Draw"

STOP

OUTPUT:



```

IDLE Shell 3.13.6
File Edit Shell Debug Options Window Help
| X |
|---|
O | | X
|---|
| | O
Player X Enter position (1-9): 5
| X |
|---|
O | X | X
|---|
| | O
Player O Enter position (1-9): 8
| X |
|---|
O | X | X
|---|
| O | O
Player X Enter position (1-9): 7
| X |
|---|
O | X | X
|---|
X | O | O
Player O Enter position (1-9): 1
O | X |
|---|
O | X | X
|---|
X | O | O
Player X Enter position (1-9): 3
O | X | X
|---|
O | X | X
|---|
X | O | O
Player X Wins!
>>>

program.py - C:/Users/SANIA/AppData/Local/Programs/Python/Python313/progeam.py (3.13.6)
File Edit Format Run Options Window Help
print(board[6], '|', board[7], '|', board[8])

def check(player):
    wins = [
        [0,1,2],[3,4,5],[6,7,8],
        [0,3,6],[1,4,7],[2,5,8],
        [0,4,8],[2,4,6]
    ]
    for w in wins:
        if board[w[0]] == board[w[1]] == board[w[2]] == player:
            return True
    return False

player = 'X'

for turn in range(9):
    display()
    pos = int(input("Player " + player + " Enter position (1-9): "))

    if board[pos] == ' ':
        board[pos] = player
    else:
        print("Position already taken")
        continue

    if check(player):
        display()
        print("Player", player, "Wins!")
        break

    if player == 'X':
        player = 'O'
    else:
        player = 'X'

else:
    display()
    print("Game Draw")

```

Result

The Tic Tac Toe game was successfully implemented in Python. The program allows two players to play alternately, correctly detects the winner, and declares a draw when no winning condition is achieved.

These are concise, lab-record-friendly Aim, Algorithm, Pseudocode, and Result sections suitable for practical examinations.